

National University of Singapore

CS2109S—Introduction to AI and Machine Learning

Midterm Assessment

Complete Context, Questions, and Answers

Semester 2, 2025/2026

Time allowed: 2 hours

	Number of questions	Total marks
Uninformed Search	6	9
Informed Search	4	8
Local Search	1	4
Adversarial Search	7	10
Decision Trees	6	8
Linear/Logistic Regression	6	12
Total	30	50

Part 1: Uninformed Search

Context

The Network Activation Problem

You are a system administrator managing a network of N servers. The network is defined by a fixed set of connections (a graph), where some servers are connected to others.

Each server can be in one of two states: **Offline (0)** or **Online (1)**.

Initially, servers can be in various configurations, such as some being online and others offline. Your goal is to get **all** servers to the **Online** state. However, the power grid is wired strangely: you cannot switch a single server on or off in isolation. When you toggle the power switch of any specific server, it flips the state of that server **AND** flips the state of every server directly connected to it.

Example: Suppose that Server A, B, C are Online, and D is Offline. If Server A is connected to Server B and D, and you toggle Server A:

- Server A flips (Online $\rightarrow$ Offline).
- Server B flips (Online $\rightarrow$ Offline).
- Server C is unchanged (Online).
- Server D flips (Offline $\rightarrow$ Online).

Problem Formulation

- **States:**
 - A binary vector of length N representing the status of every server (e.g., $[1,0,0,1, \dots]$).
- **Initial State:**
 - A specific given configuration of the network (e.g., all servers are Offline).
- **Actions:**
 - **Toggle(i):** Actuate the switch for server i (where i is any server from $1 - N$).
- **Transition Function:**
 - Applying **Toggle(i)** changes the state of server i to its opposite ($0 \rightarrow 1, 1 \rightarrow 0$).
 - Simultaneously, for every server j that is a neighbor of i (connected by a wire), the state of server j is changed to its opposite.
- **Goal Test:**
 - All servers are in state **Online (1)**. (State = $[1,1,1, \dots, 1]$).
- **Path Cost:**
 - Each Toggle action costs 1.

Notes:

- A solution may not be optimal.
- An optimal solution is the least-cost solution (fewest toggles).

Analyze the search formulation above and answer questions 1A-1F.

Questions

1A. [1 mark] Is it true that the search formulation results in a state space where a state **is reachable through more than one sequence of actions**? Note: we do not care about the search algorithms in this question since we are asking about the state space, not the search tree.

- a. Yes
- b. No

1B. [1 mark] Does the search formulation result in **a unique optimal path** if there is one?

- a. Yes
- b. No

1C. [1 mark] For an arbitrary fixed graph and initial state, is a solution **guaranteed** to exist?

- a. Yes
- b. No

1D. [2 marks] Suppose that you use search (without visited memory). Which search algorithm(s) always terminate? Select all that apply.

- a. Queue-based search (i.e., BFS, UCS)
- b. Depth-First Search (DFS)
- c. None of the above

1E. [2 marks] Which of the following search (without visited memory) algorithm(s) can we employ such that the search **always finds an answer** (valid solution) if a solution exists? Select all that apply.

- a. Breadth-First Search (BFS)
- b. Depth-First Search (DFS)
- c. Uniform-Cost Search (UCS) with constant cost $C > 0$
- d. Depth-Limited Search (DLS) with DFS and max-depth N
- e. None of the above

1F. [2 marks] Suppose that we use **search with visited memory**. Which of the following search algorithm(s) is/are the best for the problem? Select all that apply.

Best means the algorithm(s) should be complete, optimal, efficient (in terms of big O worst-case space and time complexity), and aware if there is no solution.

- a. Depth-First Search (DFS)
- b. Uniform-Cost Search (UCS) with constant cost $C > 0$
- c. Depth-Limited Search (DLS) with BFS and max-depth N
- d. Iterative Deepening Search (IDS) with DFS
- e. None of the above

Solution

1A. a

Yes. Since toggle operations commute (order doesn't matter) and are self-inverse, the same state can be reached by different sequences of actions.

Example: Starting from state S , $\text{Toggle}(A) \rightarrow \text{Toggle}(B)$ and $\text{Toggle}(B) \rightarrow \text{Toggle}(A)$ both reach the same state, but via different action sequences. More generally, any permutation of a set of toggles leads to the same state.

1B. b

No. As shown in 1A, since toggle operations commute, any permutation of the optimal set of toggles is also an optimal path with the same cost. So there can be multiple optimal paths of the same length.

1C. b

Consider $N = 2$ servers connected to each other, initial state = $[1, 0]$, goal state = $[1, 1]$.

From any state, you can only ever reach one other state: $[1, 0] \leftrightarrow [0, 1]$. The states $[0, 0]$ and $[1, 1]$ are completely unreachable.

Since the goal $[1, 1]$ is unreachable, no solution exists.

1D. c

None of the above.

Without visited memory, a search can revisit states infinitely. Since Toggle is self-inverse, an algorithm can cycle (e.g., $\text{Toggle}(A), \text{Toggle}(A), \text{Toggle}(A) \dots$ forever).

- If **no solution exists**, BFS/UCS keep expanding forever and never terminate.

- DFS can follow a cycle forever (e.g., keep toggling the same server) and never terminate.

1E. a, c, d

Key properties of the problem:

- State space is finite with 2^N states, so with visited memory all algorithms terminate and are solution-aware
- All toggle actions have uniform cost, so path cost = number of toggles
- Any solution (if one exists) can be reduced to at most N toggles, since toggling the same server twice cancels out — so the optimal solution always lies at depth $\leq N$

BFS: BFS explores the search tree level by level. It will find a solution at depth $d \leq N$.

DFS: Without visited memory, DFS can get trapped in an infinite cycle.

UCS (constant cost $C > 0$): Equivalent to BFS.

DLS with DFS and max-depth N : This limits the search to depth N . Since optimal solutions require at most N toggles, a depth limit of N is sufficient to find a solution if one exists. The depth bound prevents infinite cycling, guaranteeing termination.

1F. c or b,c or c,d, or b,c,d – see explanation below

DFS: Terminates and is solution-aware with visited memory, but **not optimal** — it finds any path to the goal rather than the shortest.

UCS (constant cost $C > 0$): With uniform costs, UCS behaves identically to BFS, expanding nodes in order of depth. It is complete, optimal (guaranteed to find the minimum number of toggles), and runs in exponential time and space. If we also compare the time complexity of a priority queue, we find that UCS has an extra $\log(2^N)$ factor.

DLS with BFS and max-depth N : Since any solution exists at depth $\leq N$, BFS within depth N is guaranteed to find the shallowest — and therefore optimal — solution. Time and space complexity is also exponential.

IDS with DFS: Complete and optimal in general, and normally attractive for its polynomial space usage. However, with visited memory the visited set already requires exponential space, nullifying its space advantage. It also repeats work across iterations, making it strictly worse than UCS/BFS in both time and space. **However, in terms of big O complexity, it's the same: $O(b^d)$.**

Answer: DLS with BFS and maximum depth N . If we disregard the priority queue time complexity, we can also consider UCS (constant cost $C > 0$), as we typically do in this course. We will accept both answers. We will also accept choosing or not choosing IDS because we have never explained the big O time complexity of IDS.

Correct:

- B. UCS (optional)
- C. DLS
- D. IDS (optional)

Part 2: Informed Search

Context

The Robot Shuffle Problem

(Adapted from AIAMA.)

Consider a warehouse modelled as an $n \times n$ grid. Warehouse robots are located at grid points (x, y) , where $x = 1, \dots, n$ is also called the “left to right” direction and $y = 1, \dots, n$ is also called the “bottom to top” direction.

The warehouse is populated by n robots.

Initially, let the robots be arranged in the bottom row of the grid. In other words, the i -th robot is at grid point $(i, 1)$. The task is to move all the robots to the top row while reversing their order. The top row is where $y = n$. In other words, the task is to move the i -th robot from $(i, 1)$ to $(n - i + 1, n)$.

The following defines the possible actions for the n robot problem. At each time step, an action is defined as every one of the robots performing one of the following: move up, move down, move left, move right, or stay put (i.e., no move). For any single robot that stays put, a single adjacent robot can jump over it. A hop of a single robot is only possible over a single robot. Two robots cannot occupy the same square. Each time step comes with cost 1.

Analyze the problem above and answer questions 2A-2D. Consider that the questions impose further assumptions.

The heuristics for the questions 2A-2C are given as follows.

2A & 2B

1. $h1_i := 1$ if the position (x_i, y_i) is not the position $(n - i + 1, n)$, and 0 otherwise.
2. $h2_i := \sqrt{(x_i - n + i - 1)^2 + (y_i - n)^2}$
3. $h3_i := |x_i - n + i - 1| + |y_i - n|$
4. $h4_i := \left(\sqrt{|x_i - n + i - 1|} + \sqrt{|y_i - n|} \right)^2$

2C.

1. $hC1_i := \frac{1}{4}(|x_i - n + i - 1| + |y_i - n|)$
2. $hC2_i := \frac{1}{2}(|x_i - n + i - 1| + |y_i - n|)$
3. $hC3_i := |x_i - n + i - 1| + |y_i - n|$

4. $hC4_i := 2 (|x_i - n + i - 1| + |y_i - n|)$
5. $hC5_i := 4 (|x_i - n + i - 1| + |y_i - n|)$

Questions

2A: [2 marks]

Consider a single robot i at position (x_i, y_i) . For this question, there are no other robots. Consider the problem of moving the robot to its goal location. Select all the admissible heuristics for the number of moves to the goal location that apply.

- a) $h1_i$
- b) $h2_i$
- c) $h3_i$
- d) $h4_i$
- e) None of the above.

2B: [2 marks] Consider a single robot i at position (x_i, y_i) . There are $n - 1$ other robots and their positions are fixed. The action only moves robot i . Select all the admissible heuristics for the number of moves to the goal location that apply.

- a) $h1_i$
- b) $h2_i$
- c) $h3_i$
- d) $h4_i$
- e) None of the above.

2C: [2 marks] Consider the same setting as Question **2B**. Select all the consistent heuristics for the number of moves to the goal location that apply.

- a) $hC1_i$
- b) $hC2_i$
- c) $hC3_i$
- d) $hC4_i$
- e) $hC5_i$
- f) None of the above.

2D: [2 marks] Consider the problem of shuffling n robots as described in the Context. Consider an admissible heuristic $h3_i$ for the single-robot setting as in Question **2A**. Select all the admissible heuristics for the n -robots problem that apply.

- a) Sum: $\sum_{i=1}^n h_i$.
- b) Max: $\max \{h_1, \dots, h_n\}$.
- c) Min: $\min \{h_1, \dots, h_n\}$.
- d) None of the above.

Solution

2A. a, b, and c.

The true cost of the single-robot problem is the Manhattan distance $|x_i - n + i - 1| + |y_i - n|$. We have that $h1_i \leq h2_i \leq h3_i$, and heuristics 1, 2, and 3 are admissible. Heuristic $h4_i$ is not admissible, since if the robot is currently at point $(n - i, n - 1)$ it takes 2 steps to reach the goal, but the heuristic will give $(1 + 1)^2 = 4$.

2B. a.

Another robot may be at the target position. In that case the true cost is infinity and all heuristics are admissible. However, admissibility must hold for all states. Assume from now that none of the $n-1$ robots is at the target position. Assume the main robot is at $(n - i + 1, n - 2)$, so it is in the goal left to right direction, but not yet at the goal in the top to bottom direction. The heuristic estimates are $h1_i = 1, h2_i = h3_i = h4_i = 2$. However, the true cost is 1 since the robot can jump over the other robot with cost 1 to reach the goal. Only $h1_i$ is admissible.

2C. ab.

We can rule out the inadmissible heuristics $hC3_i, hC4_i, hC5_i$.

The main robot can jump over another robot at cost 1 to get closer to the goal by two positions. Considering the y direction only: $hC_i(x_i, y_i) \leq 1 + h_i(x_i, y_i + 2) \leftrightarrow hC_i(x_i, y_i) - h_i(x_i, y_i + 2) \leq 1$. This inequality is satisfied by $hC1_i, hC2_i$. Recall that only a jump over a single robot is possible.

2D. c.

Despite that any robot i could help another robot to achieve the goal faster, that robot i must at least go to its goal position which is $h3_i$ away. Hence, the Min of the admissible heuristic is admissible.

A counterexample for the Max is given as follows, see picture. Here the max is 3 as robot 4 is 3 steps away from the Goal G4. However, the true cost is 2: First, robot 4 jumps over robot 3 at cost 1. Then Robot 3 and 4 move to their respective goals at cost 1. Hence the total cost is 2.

Recall again that the context states that at one timestep every robot can move. Hence in a single time step both robots move at cost 1.

 5	G4	G3	 2	 1
		 3		
		 4		

Part 3: Local Search

Context

Refer to the description of **The Network Activation Problem**.

Questions

3A. [4 marks] Given the problem, which of the following local search formulation(s) is/are reasonable for hill climbing algorithm taught in lecture? Select all that apply.

Notes:

- In this context, we consider the formulation reasonable if hill-climbing with an infinitely large number of random restarts can return the optimal sequence of actions for the problem if the solution exists.
- At every restart, the initial state and the output of the successor function may differ from the previous run if there is randomness involved.
- Hill climbing picks a successor based on the highest evaluation function's value

a)

- **State:** A binary vector of length N , representing the current On/Off status of each server.
- **Initial State:** The given initial configuration of the network.
- **Goal Test:** Is the vector $[1, 1, \dots, 1]$ all Online?
- **Evaluation Function:** Number of servers currently Online.
- **Successors:** Generate all states reachable by toggling exactly one switch from the current state.

b)

- **State:** A binary vector of length N , where the i -th bit represents whether switch i is included in the solution set.
- **Initial State:** A random binary vector of length N .
- **Goal Test:** Does applying this set of switches result in all servers being Online?
- **Evaluation Function:** $(-A \times (\text{Number of Offline servers resulting from this set}) - B \times (\text{Number of 1s in the state vector}))$, where A and B are parameters, $A \gg B > 0$.
- **Successors:** Generate all vectors that differ from the current vector by exactly one bit (add or remove one switch).

c)

- **State:** A list of integers representing the sequence of switches pressed so far (e.g., $[3, 1, 5]$).
- **Initial State:** An empty list $[]$.
- **Goal Test:** Does applying this sequence of switches result in all servers being Online?
- **Evaluation Function:** Number of servers currently Offline.
- **Successors:** Append any integer $k \in [1, N]$ to the end of the current list.

d)

- **State:** A binary vector of length N , where the i -th bit represents whether switch i is included in the solution set.
- **Initial State:** The given initial configuration of the network.
- **Goal Test:** Does applying this set of switches result in all servers being Online?
- **Evaluation Function:** 0 if all servers are Online; otherwise $-\infty$.
- **Successors:** Generate all vectors that differ from the current vector by exactly one bit.

e) None of the above

Solution

3A. b

a)

Only tracks the current server state, not which switches were pressed. When the algorithm terminates, it just reports the final server state.

b)

The state encodes which switches are included in the solution set (each server toggled at most once). The evaluation function penalizes both offline servers (heavily, via A) and the number of switches used (lightly, via B , with $A \gg B > 0$).

- The dominant term ($-A \times$ offline servers) drives the search toward valid solutions
- The secondary term ($-B \times$ switches used) breaks ties in favor of fewer toggles, directly encoding optimality
- Since the initial state is **random** and successors flip one bit, with infinite restarts the search can reach any binary vector of length N , including the optimal solution set
- The state space is finite (2^N), so the global optimum is reachable

Formulation is reasonable.

c)

The evaluation is the *number of offline servers* but hill climbing chooses highest value, so it is optimizing in the wrong direction.

d)

The evaluation is 0 only at goal and $-\infty$ otherwise, so hill climbing will directly terminate in one step if the goal is >1 step away since all states are equally bad, i.e., value = $-\infty$. Since the initial state is fixed based on the problem, infinite restarts will not help solve problems whose goal is >1 step away.

Part 4: Adversarial Search

Minimax

Context

Sector Control

Two commanders, A and B, are fighting for control over a 1D grid map with bounds ± 6 . A flag starts at position -1 .

- **Commander A** is the **Max** player. Their goal is to move the flag to the positive end.
- **Commander B** is the **Min** player. Their goal is to move the flag to the negative end.

The valid bounds of the grid map shrink over time. Every turn, the bounds shrink by **1**, until they reach a minimum of ± 1 .

Players alternate turns. On their turn, a player can choose one of the following actions, provided the move does not violate the **No-repeat Rule**:

1. **Push**: Move the flag **1 step** in their target direction (A adds 1, B subtracts 1).
2. **Boost**: Move the flag **2 steps** in their target direction (A adds 2, B subtracts 2).
3. **Invert**: Multiply the current position by **-1**. *Constraint*: This move is only available if the current position is **non-zero**.

The No-repeat Rule: A player **cannot** make a move that returns the flag to the exact position it occupied at the start of the previous turn. For instance, if the current position is 0, and Commander A executes a Push move, the position changes to 1. However, Commander B is not permitted to Push again because it would revert the position to 0.

Game End & Utility:

- **Victory**: If the flag lands on or beyond the positive current bound, Commander A wins (Utility: **+10**). If the flag lands on or beyond the negative current bound, Commander B wins (Utility: **-10**). No more move is possible.
- **Stalemate**: If a player has no valid moves, the game ends. The utility is the **current integer position of the flag**.

We will use **Minimax with cutoff** at **Depth 4** as our algorithm. Heuristic: At the cutoff depth, the heuristic value is the **current integer position of the flag**. (e.g., if the game cuts off at position -3 , the heuristic value is -3).

Notes:

- The value of a state is evaluated from the perspective of the max player.
- Commander A goes first.
- The game tree is *tree* (a child can only have one parent).
- Root node is at Depth 0.

Terminologies:

- Terminal states: states where the game ended (not cutoff)
- Terminal nodes: nodes that contain terminal states

Questions

4A. [1 mark] How many terminal nodes are there in the game tree generated by the algorithm specified in the context?

4B. [1 mark] How many nodes in the **full** game tree generated by Minimax without cutoff exceed the cutoff depth specified in the context? Note: please write “infinity” if the depth is infinite.

4C. [1 mark] What is the game tree's maximum depth for Minimax without cutoff? Note: please write “infinity” if the depth is infinite.

4D. [1 mark] Suppose Commander A Boost the flag to position **+1** on their first turn. Which commander can win the game if they both play optimally in the subsequent turns until the terminal state?

- A
- B
- Game is draw
- Game continues indefinitely
- None of the above

4E. [1 mark] Let the following be sequences of play from the first turn (i.e., beginning of the game). Which sequence(s) give(s) the highest value to Commander A according to the output of the algorithm specified in the context? Select all that is/are true.

- A: Push → B: Boost → A: Invert → B: Boost
- A: Boost → B: Push → A: Boost → B: Invert
- A: Push → B: Boost → A: Invert → B: Push
- A: Invert → B: Push → A: Boost → B: Push
- None of the above

4F. [1 mark] Which of the following action(s) can the Commander A take on their first turn (i.e., beginning of the game) to gain the maximum value based on the output of the

algorithm specified in the context? Assume all players play optimally in the subsequent turns according to the algorithm specified in the context. Select all that is/are true.

Note: If there are multiple actions that Commander A can take, select all the actions. If all actions have the same utility, do not select any action and select option d.

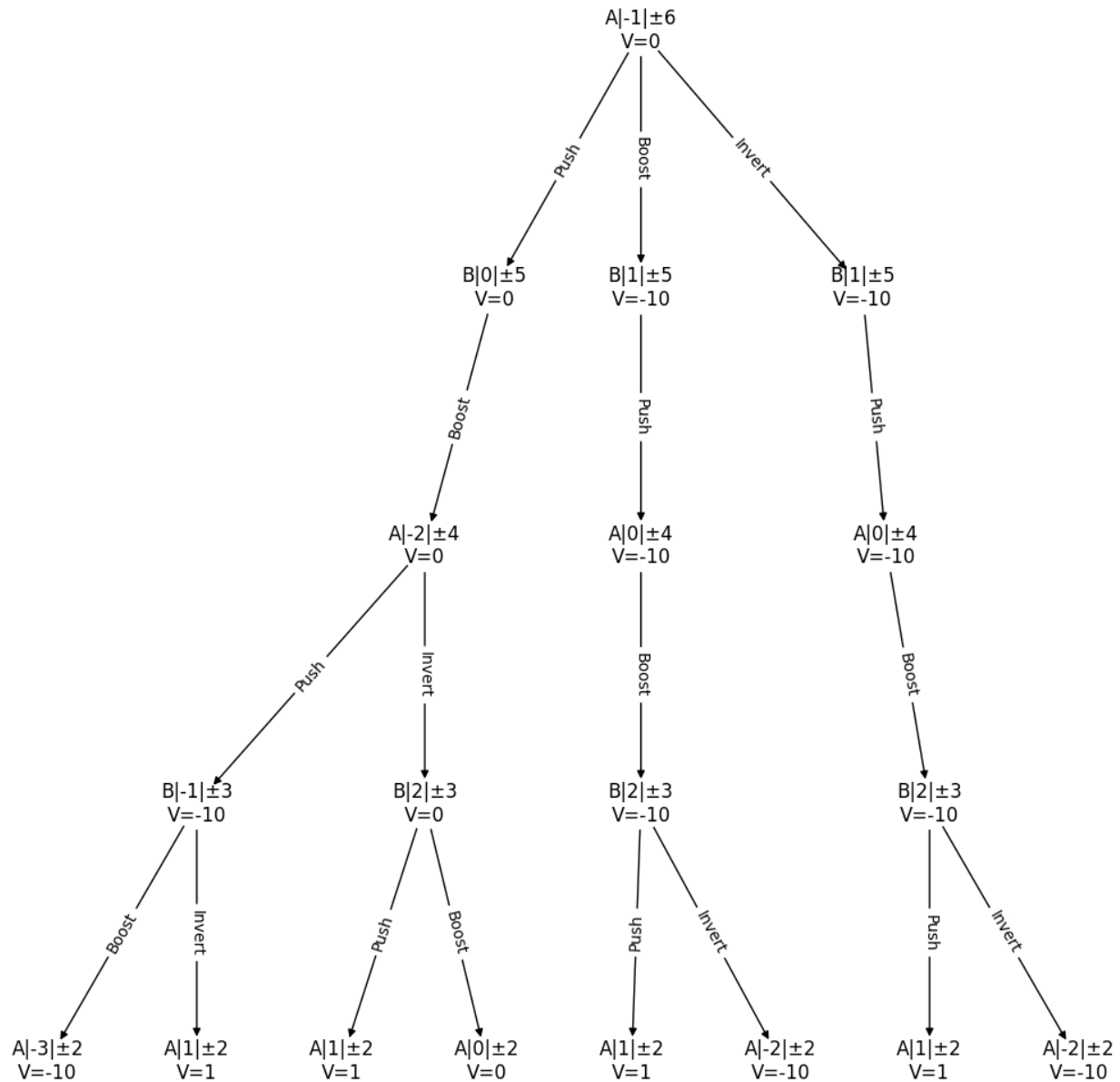
- a. Push
- b. Boost
- c. Invert
- d. All actions result in the same utility
- e. None of the above

Solution

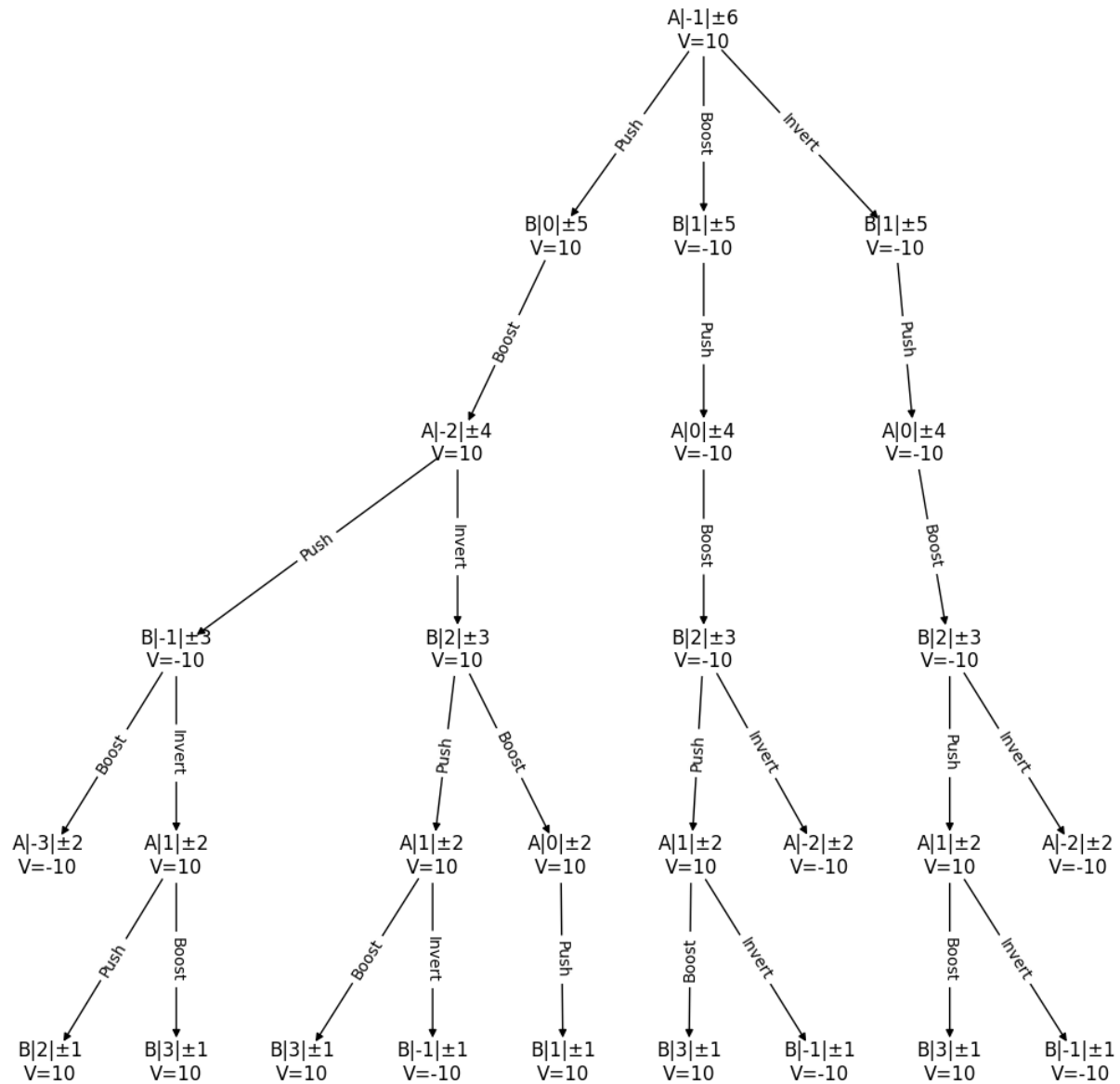
During the exam, we clarified two things:

- Bound shrink occurs simultaneously with the move. For instance, if the current bound is 4 and A move to position 3: the bound shrinks to 3, hence player A wins.
- Bound shrink occurs from both sides, e.g., $+6$ to $+5$.

Minimax with cutoff depth 4:



Full Minimax:



4A. 3

4B. 9

4C. 5

4D. b

Notice that the question asks for the outcome when the game is played until termination (not cutoff).

“... if they both play optimally in the subsequent turns until the **terminal state**”

As seen on the full minimax game tree above, Commander B wins.

4E. c, d

Based on the minimax game tree with cutoff 4 above:

a) A: Push → B: Boost → A: Invert → B: Boost: 0

b) A: Boost → B: Push → A: Boost → B: Invert: -10

c) A: Push → B: Boost → A: Invert → B: Push: 1

d) A: Invert → B: Push → A: Boost → B: Push: 1

c and d give the highest value of 1.

4F. a

Based on the minimax game tree with cutoff 4 above:

Push → 0 (maximum value)

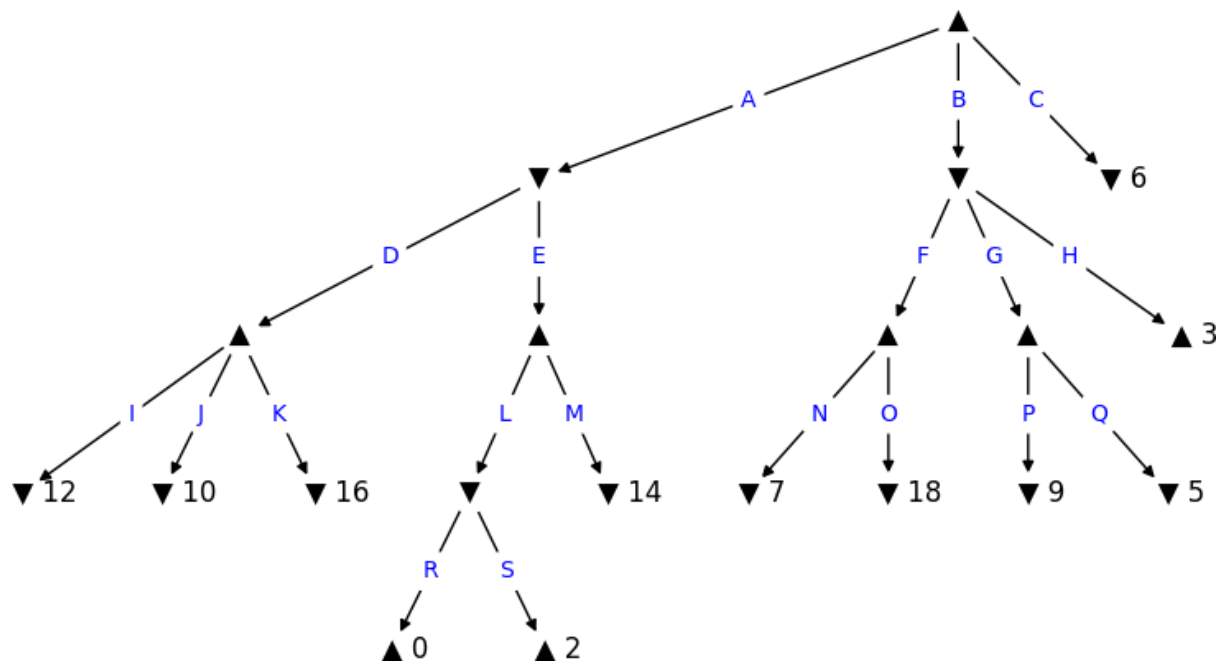
Boost → -10

Invert → -10

Alpha-Beta Pruning

Context

Consider the following game tree for a two-player game where alpha-beta pruning is applied.



Note: The symbol ▲ represents the max player's turn, while ▼ indicates the min player's turn.

Answer question 4G.

Questions

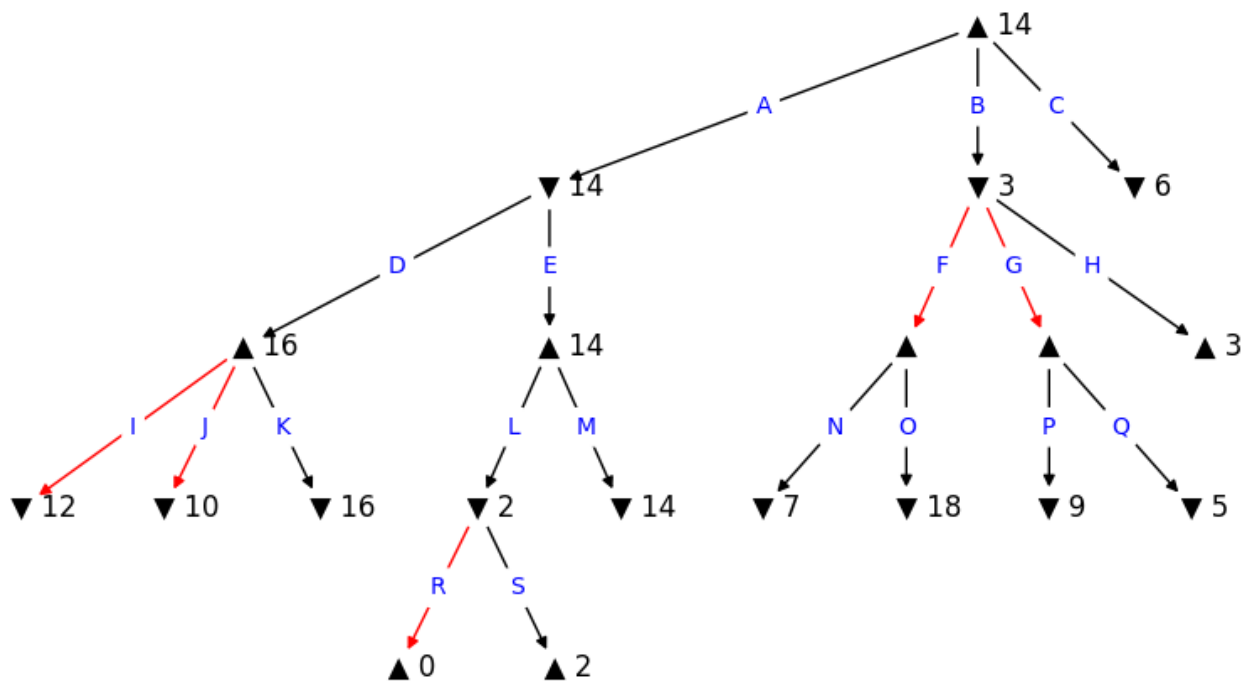
4G. [4 marks] Suppose we traverse this tree using (depth-first) alpha-beta pruning from **right to left**. Select **all** the link(s) that would be pruned by alpha-beta pruning algorithm. Select only the links that are directly pruned and not those that are indirectly pruned because they are in a subtree of a pruned link.

Which of the following link(s) is/are pruned? Select all that is/are true.

- a. A
- b. B
- c. C
- d. D
- e. E
- f. F

- g. G
- h. H
- i. I
- j. J
- k. K
- l. L
- m. M
- n. N
- o. O
- p. P
- q. Q
- r. R
- s. S
- t. None of the above

Solution



4G. f, g, i, j, r

Part 5: Decision trees

You work at a real-estate company and have been tasked with deciding on investing in an apartment. Leveraging your expertise, you create a decision tree based on previous investing data given in Table 1.

	Pool	Quality	Location	Decision
1	Yes	Mid	A	Yes
2	Yes	High	B	No
3	No	Mid	C	Yes
4	Yes	High	B	Yes
5	No	High	A	No
6	Yes	Mid	D	Yes
7	No	Low	C	No
8	Yes	Mid	B	Yes
9	No	High	C	No
10	Yes	High	D	No
11	No	Low	B	No
12	Yes	High	C	Yes

Consider the decision tree learning algorithm using information gain.

In the case of ties, the preference is Location > Quality > Pool. The default value at the initial DTL call is “No”. The majority decision function outputs “Yes” in case of a tie.

Based on this description, answer questions 5A-5F.

Notes:

- If a decision tree does not check an attribute, it is a zero-depth decision tree.
- The entropy for a given probability distribution p_i , for $i = 1, \dots, n$ is given by:

$$Entropy = - \sum_{i=1}^n p_i \log_2(p_i).$$

- The conditional entropy of a random variable Y given X is given by:

$$H(Y|X) = \sum_{x \in X} P(X = x) H(Y|X = x).$$

- Remember the log is in base 2!
- Values for the log: $\log_2(1) = 0$; $\log_2(2) = 1$; $\log_2(3) = 1.585$; $\log_2(4) = 2$; $\log_2(5) = 2.322$; $\log_2(6) = 2.585$; $\log_2(7) = 2.807$; $\log_2(8) = 3$; $\log_2(9) = 3.170$; $\log_2(10) = 3.322$.
- For rounding to two decimal places, round to the closest number with two decimal places. For example, 1.234 is rounded to 1.23 and 1.237 is rounded to 1.24.

Questions

5A: [1 mark] What is the entropy of the Decision (yes/no) given the data in the table, rounded to two decimal places?

5B: [1 mark] What is the information gain of selecting “Location” as the root node of the Decision (yes/no) in the table, rounded to two decimal places?

5C: [1 mark] What is the information gain of selecting “Quality” as the root node of the Decision (yes/no) in the table, rounded to two decimal places?

5D: [1 mark] What is the information gain of selecting “Pool” as the root node of the Decision (yes/no) in the table, rounded to two decimal places?

5E: [2 marks] Build the decision tree given the context information up to maximum depth 2. Select the attributes that are **not** tested anywhere in that decision tree. Select all that apply.

- Location.
- Quality.
- Pool.
- All attributes are tested.

5F: [2 marks] According to the decision tree built in Question **5E**, consider the Decision for the following examples.

Select the example(s) with a Yes decision. Select all that apply.

- Pool: Yes. Quality: High. Location: A.
- Pool: No. Quality: High. Location: B.
- Pool: Yes. Quality: Mid. Location: C.
- Pool: No. Quality: Low. Location A.
- None of the above.

Solution

5A. 1

Explanation: $-\frac{6}{12} \log_2 \left(\frac{6}{12} \right) - \frac{6}{12} \log_2 \left(\frac{6}{12} \right) = 1.$

5B. 0

The conditional entropy when splitting according to Location is $H(Y|L) = \frac{2}{12} H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{4}{12} H\left(\frac{2}{4}, \frac{2}{4}\right) + \frac{4}{12} H\left(\frac{2}{4}, \frac{2}{4}\right) + \frac{2}{12} H\left(\frac{1}{2}, \frac{1}{2}\right) = 1.$ Hence the information gain is 0.

5C. 0.54

The conditional entropy when splitting according to Quality is $H(Y|Q) = \frac{6}{12} H\left(\frac{2}{6}, \frac{4}{6}\right) + \frac{4}{12} H\left(\frac{0}{4}, \frac{4}{4}\right) + \frac{2}{12} H\left(\frac{0}{2}, \frac{2}{2}\right) = \frac{1}{2} * 0.918 = 0.459.$ Hence the information gain is $1 - 0.459 = 0.541.$

5D. 0.20

The conditional entropy when splitting according to Pool is $H(Y|P) = \frac{7}{12} H\left(\frac{2}{7}, \frac{5}{7}\right) + \frac{5}{12} H\left(\frac{4}{5}, \frac{1}{5}\right) = 0.80.$ Hence the information gain is $1 - 0.80 = 0.20.$

5E. c or d.

Select attribute Quality. For value = High we obtain six samples. The other values are decisive.

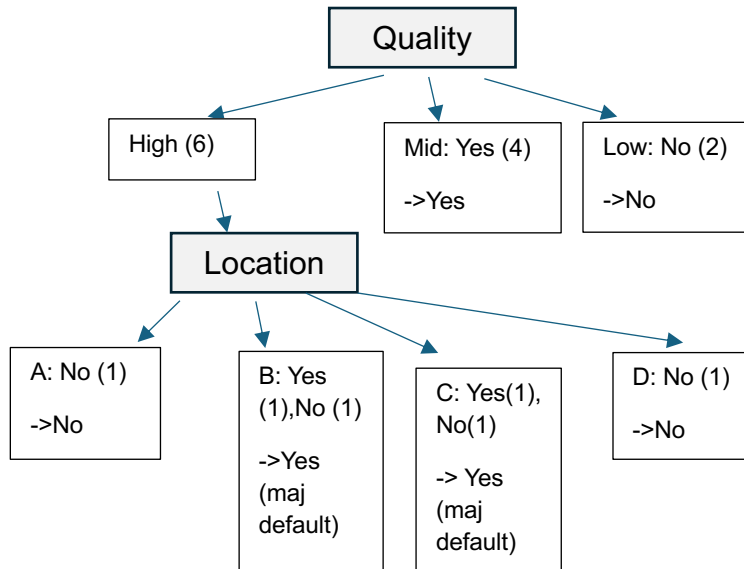
	Pool	Quality	Location	Decision
2	Yes	High	B	No
4	Yes	High	B	Yes
5	No	High	A	No
9	No	High	C	No
10	Yes	High	D	No
12	Yes	High	C	Yes

The base entropy is $H(Y| \text{Quality=High}) = H\left(\frac{2}{6}, \frac{4}{6}\right) = 0.918.$

The conditional entropy when splitting according to Location is $H(Y| \text{Quality=High}, L) = \frac{1}{6} H(1,0) + \frac{2}{6} H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{2}{6} H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{1}{6} H(0,1) = \frac{4}{6} = 0.66.$ Hence, the information gain is 0.258.

The conditional entropy when splitting according to Pool is $H(Y|Quality=High,P) = \frac{4}{6} H\left(\frac{2}{4}, \frac{2}{4}\right) + \frac{2}{6} H(0,1) = 0.66$. Hence, the information gain is 0.258.

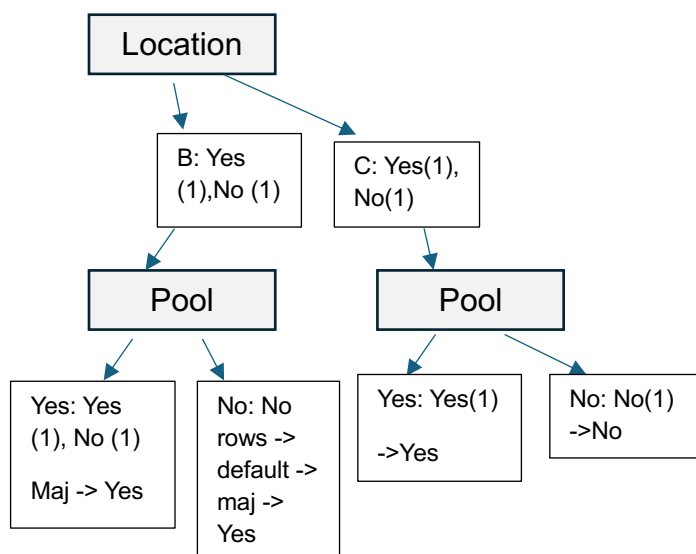
The decision tree is as follows:



By the problem statement, if a decision tree does not check an attribute, it is a zero-depth decision tree. This decision tree is depth 2; it is the intended solution decision tree.

Alternative assumption:

We allow the interpretation “First attribute is checked at depth zero”. Under that interpretation, we continue the location B, C cases.



5E. bc.

- a) Pool: Yes. Quality: High. Location: A. -> No
- b) Pool: No. Quality: High. Location: B. -> Yes
- c) Pool: Yes. Quality: Mid. Location: C. -> Yes
- d) Pool: No. Quality: Low. Location A. -> No

Part 6: Linear regression and logistic regression.

Round to two decimal places, i.e., round to the closest number with two decimal places. For example, 1.234 is rounded to 1.23 and 1.237 is rounded to 1.24.

Context for Questions 6A-C.

Consider the linear regression model $h_w(x) = w_0 + w_1x_1$.

Consider the MSE loss function $J_{MSE}(w) = \frac{1}{2n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)})^2$.

Consider the following training data.

Training point	x_1	y
$x^{(1)}$	-1	1
$x^{(2)}$	1	-1
$x^{(3)}$	2	2

Consider the initial weights for linear regression of $w = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} 4 \\ -2 \end{bmatrix}$.

Context for Questions 6D-F.

No context

Questions

6A: [2 marks] Using the given data, construct the hypothesis $h_w^{poly}(x)$ of polynomial regression of degree 2. The initial weight value for the additional polynomial feature shall be

$w_2 = 1$. Evaluate the hypothesis and the corresponding loss function for the data points.
Fill in the following blanks:

$$h_w^{poly}(x^{(1)}) \text{ [BLANK1]}$$

$$h_w^{poly}(x^{(2)}) \text{ [BLANK2]}$$

$$h_w^{poly}(x^{(3)}) \text{ [BLANK3]}$$

$$J_{MSE}(w) \text{ [BLANK4]}$$

6B: [2 marks] You would like to learn the hypothesis $h_w^{poly}(x)$. Perform a single update to the vector $\begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$, where $w_2 = 1$, using the **gradient descent (GD)** update rule with the

learning rate $\gamma = 1/2$. Fill into the blank the resulting weights $\begin{bmatrix} w_0' \\ w_1' \\ w_2' \end{bmatrix}$ as follows: $\begin{bmatrix} \text{Blank 1} \\ \text{Blank 2} \\ \text{Blank 3} \end{bmatrix}$

6C: [2 marks] The same context as for Questions **6A** and **6B** applies. Select potential problems when training $h_w^{poly}(x)$ and when using $h_w^{poly}(x)$ with new data. Consider the data that are available and new data that the model will be used with. Select all that apply.

- a) In the context, there is not a sufficient amount of data points for the described polynomial regression.
- b) The new feature is linearly dependent on the original feature.
- c) The MSE is only convex and not strictly convex.
- d) You receive the knowledge about the possible range for $x_1 \in [-100, 100]$.
- e) The negative gradient of MSE points away from the minimum of the MSE.
- f) None of the above.

6D: [2 marks] Let hospital patients be described by two features x_1 and x_2 . The hospital has found a decision boundary for **healthy** and **not-healthy** patients given by $x_2 = 1 + 2x_1$, where the patient is **not-healthy** if $x_2 \geq 1 + 2x_1$. You want to find the logistic regression model given by three weights $\begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$, where w_0 is for the offset, w_1 is for the feature x_1 , and w_2 is for the feature x_2 . The logistic model with threshold 0.5 should output 1 if the patient is **not-healthy**. Fill in the blanks with the weights of the logistic regression model.

$$w_0 = \text{[BLANK1]}$$

$$w_1 = [BLANK2]$$

$$w_2 = [BLANK3]$$

6E: [2 marks] Consider the following customer data. The customers are described by the following features and a target variable.

x_1	x_2	x_3	y
1000	0.1	0.4	$y^{(1)}$
3400	0.5	2.0	$y^{(2)}$
500	0.25	1.0	$y^{(3)}$
1900	0.8	3.2	$y^{(4)}$

The values $y^{(1)}, \dots, y^{(4)}$ are either 0 or 1.

A company uses the logistic regression model $\sigma(w_1x_1 + w_2x_2 + w_3x_3)$, without offset. You are asked to submit alternative models including a recommendation to retrain the model or not. Consider the scenario where retraining is expensive, hence a retraining recommendation should only be made when necessary. Also, the alternative model should be at least as expressive as the current model.

Note that a more expressive model is a model with higher model complexity.

Select the logistic regression models and the corresponding recommendation. Select all that apply.

- a) Model: $\sigma((w_1 + 4w_2)x_2 + w_3x_3)$. Retrain: no.
- b) Model: $\sigma((w_1 + 4w_2)x_2 + w_3x_3)$. Retrain: yes.
- c) Model: $\sigma(w_1x_1 + (w_2 + 4w_3)x_2)$. Retrain: no.
- d) Model: $\sigma(w_1x_1 + (w_2 + 4w_3)x_2)$. Retrain: yes.
- e) Model: $\sigma(w_1 \frac{|x_1 - 500|}{2900} + w_2x_2)$. Retrain: no.
- f) Model: $\sigma(w_1 \frac{|x_1 - 500|}{2900} + w_2x_2)$. Retrain: yes.
- g) None of the above.

6F. [2 marks] You are given a data set that consists of images of size 1024×768 , where each pixel of every image has a grayscale value $g \in [0,1]$. You have 100 images available.

You create a feature vector x , where each feature corresponds to the greyscale value of a pixel. Additionally, each image has an associated binary target y . You plan to use a logistic regression model $h(x)$, and you aim to minimize the BCE loss to predict the target value for new images. Given these choices, select true statements about the problem and the associated training of the model?

- a) The normal equation will be the preferred method over gradient descent for training the model.
- b) The model has significantly more parameters than data points.
- c) Min-max scaling is necessary to improve the training of the model.
- d) Gradient descent will be about 0.01 times as fast as stochastic gradient descent per iteration.
- e) None of the statements are true.

Solution

6A. 7, 3, 4, 9.33.

The hypothesis is $h_w^{poly}(x) = w_0 + w_1x_1 + w_2x_1^2$. Since we have only a single feature and degree 2, there is only one possible way to write the model. Update the table as follows:

Training point	x_1	x_1^2	y
$x^{(1)}$	-1	1	1
$x^{(2)}$	1	1	-1
$x^{(3)}$	2	4	2

To be able to evaluate the loss function, we need weights values. However, we have “initial weights” for the linear model $\begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$ and the “initial weight value” w_2 for the polynomial feature.

Hence, we take these values to evaluate the hypothesis. I.e., $\begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 4 \\ -2 \\ 1 \end{bmatrix}$.

$$h_w(x^{(1)}) = 4 - 2 * (-1) + 1 * 1 = 7$$

$$h_w(x^{(2)}) = 4 - 2 * 1 + 1 * 1 = 3$$

$$h_w(x^{(3)}) = 4 - 2 * 2 + 1 * 4 = 4$$

We evaluate the “corresponding” loss function, i.e., the loss function using $h_w^{poly}(x)$.

$$J_{MSE}(w) = \frac{1}{6} [(7-1)^2 + (3+1)^2 + (4-2)^2] = \frac{1}{6} (36 + 16 + 4) = 9.33.$$

That we must use the loss function using $h_w^{poly}(x)$ is clear from the word “corresponding”, and from the fact that MSE (and other) loss functions are conceptually always dependent of the model under consideration. Hence the definition in the context $J_{MSE}(w)$ does not fix the loss function to the linear regression model.

6B. 2, -2.33, -2.

The gradient vector is:

$$\begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} = \frac{1}{3} [(h_w^{poly}(x^{(1)}) - y^{(1)})x^{(1)} + (h_w^{poly}(x^{(2)}) - y^{(2)})x^{(2)} + (h_w^{poly}(x^{(3)}) - y^{(3)})x^{(3)}] =$$

$$\frac{1}{3} \left[(6) \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} + (4) \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + (2) \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} \right] = \frac{1}{3} \begin{bmatrix} 12 \\ 2 \\ 18 \end{bmatrix} = \begin{bmatrix} 4 \\ 2/3 \\ 6 \end{bmatrix}.$$

Using the initial weights as in **6A**, we arrive at:

$$\begin{bmatrix} w_0' \\ w_1' \\ w_2' \end{bmatrix} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} - \gamma \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} = \begin{bmatrix} 4 \\ -2 \\ 1 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 4 \\ 2/3 \\ 6 \end{bmatrix} = \begin{bmatrix} 2 \\ -2.33 \\ -2 \end{bmatrix}.$$

6C: a or d or ad or f.

b) The new feature is not linearly dependent on the original feature, $x_2 = x_1 * x_1$.

c) MSE is strictly convex here. Also, only convex would not be a problem.

d) **May lead to a problem.** You receive the knowledge about the possible range for $x_1 \in [-100, 100]$. This **can be** a problem for training with more data points, because feature 2 is on very different scale as feature 1, i.e., may be 100 times of feature 1. Gradient descent may converge very slowly. Also, the current training set does not include points in that range, hence the prediction may be very wrong for data points with, e.g., feature 1 = 100.

e) Not correct. The negative gradient points to the minimum. Not a problem.

Update: We allow the following options.

- a (optional)
- d (optional)

- f when suitable.

Explanation:

a: Indeed, under a machine learning lens, the 3 data points, while allowing a fit, will probably lead to overfitting. This can be a potential problem when using the model with new data.

d: We decided that the wording can be assumed to be too vague to imply a problem. While indeed a large range for the feature may pose a problem (due to features with different scales and training example not seen in that range), Option d is not stated clear enough under the assumption that we require stronger evidence.

6D. $[-1, -2, 1]$ or $c[-1, -2, 1]$, where c is a positive number.

Start from: $x_2 \geq 1 + 2x_1$ for non-healthy patients. This equation implies that a patient is not-healthy if $x_2 - 2x_1 - 1 \geq 0$. The logistic function is $\sigma(z) \geq 0.5$ for $z \geq 0$. The logistic model $\sigma(x_2 - 2x_1 - 1)$ with threshold 0.5 will output 1 for non-healthy patients. Hence, the weights of a possible logistic model are given by $w_0 = -1, w_1 = -2, w_2 = 1$. Any other positive scaling of this weight vector also works.

6E: cf.

Feature x_3 is 4 times feature x_2 . Hence, we can remove it at no loss of model complexity.

a) Model: $\sigma((w_1 + 4w_2)x_2 + w_3x_3)$. Retrain: no.

False, the model complexity is reduced by not considering feature 1.

b) Model: $\sigma((w_1 + 4w_2)x_2 + w_3x_3)$. Retrain: yes.

False, the model complexity is reduced by not considering feature 1.

c) Model: $\sigma(w_1x_1 + (w_2 + 4w_3)x_2)$. Retrain: no.

True, no retraining needed if we combine the weights as given.

d) Model: $\sigma(w_1x_1 + (w_2 + 4w_3)x_2)$. Retrain: yes.

False, no retraining needed. From the problem statement, retraining should be avoided.

e) Model: $\sigma(w_1 \frac{|x_1 - 500|}{2900} + w_2x_2)$. Retrain: no.

False, while the model complexity is retained, we must retrain the model. Also, the feature scaling means that we must retrain the model.

- f) Model: $\sigma(w_1 \frac{|x_1 - 500|}{2900} + w_2 x_2)$. Retrain: yes.

True, while the model complexity is retained, we must retrain the model. Also, the feature scaling means that we must retrain the model.

6F. bd

- a) False, logistic regression cannot be solved via the normal equation.
- b) True, the model has $786432 + 1$ parameters and only 100 data points are given.
- c) False, while min-max scaling may achieve some improvements, it is not necessary to improve the training of the model. All the features already are on a scale of $[0,1]$.
- d) True, gradient descent uses in every iteration all the 100 data points. Hence it will be about 100 times slower than stochastic gradient descent per iteration, where SGD uses only a single data point.